You said:

from the traffic tool conversations weve had I am going to ask you to give me all of the python code for the website using drafter

ChatGPT said:

Sounds good! We can build your **Traffic Flow Visualization Website** entirely in Python using the **Drafter** library, based on the traffic tool project we've been discussing.

I'll structure the code around your original idea: a website that allows users to explore how different road types and interchange designs affect traffic flow, with visual representations.

I'll also make sure the code meets your project's requirements:

- **2 dataclasses:** A `State` dataclass with at least 4 attributes, including a list of instances of a second dataclass.
- **At least 7 routes:** Functions that handle navigation and user interactions.
- **At least 5 pages:** Including a home page, interchange selection, traffic configuration, visualization, and results.
- **At least 3 user inputs:** Using Drafter components such as `TextBox`, `SelectBox`, and `CheckBox`.
- **Traffic visualizations:** Showing different interchange layouts and how traffic moves through them.
- **Traffic analysis:** Comparing traffic volume, congestion, and interchange efficiency.

I'll aim to keep the code **appropriate for your Python course**, using straightforward functions, loops, conditionals, dataclasses, and Drafter components rather than unnecessarily advanced programming.

One important distinction: the traffic calculations will be simplified educational models, not professional traffic engineering simulations.

When you're ready, send me the specific features you want included, and I'll put together the complete Python code so the pages, routes, user inputs, and visualizations work together.

You said:

these are the things we said we would use,This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	Implementation
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	The purpose of this program will be to better visualize/understand traffic flow through different intersections depending on how dense the traffic is.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	Intersection_type: str

- This stores the type of intersection and changes when the user switches to a different intersection.

Traffic_density: int

- The amount of traffic, 1(low), 2(med), 3(high).

Vehicles: list[Vehicle]

- Stores all of the vehicles in the simulation.

Simulation_running: bool

- States whether the simulation is running or not

Simulation_step: int

- Tells how many simulation steps have happened

Second Dataclass	**Class Vehicle:**
	Vehicle_id: int
Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	

- Individual identification number

Position: int

- A number that represents the vehicles position on the route

Speed: int

- How far the vehicle can move during each simulation step

Direction: str

- The vehicles direction of travel

Waiting: bool

- Tells whether the vehicle is stopped in traffic

Routes (minimum 7) **The names of your route functions**	**Home: homepage** **Choose_intersection: intersection selection page** **Set_traffic: traffic density selection** **Simulation: traffic visualization in current intersection** **Start_simulation: starts the simulation with vehicles** **Advance_simulation: moves vehicles** **Reset_simulation: resets simulation** **Results: statistics and simulation results**
Pages (minimum 5) A brief description of the pages in your site	Homepage: purpose of program and shows how people can explore traffic flow. Intersection selection: where the user can select which intersection they want to observe Traffic settings: where users can select the density of traffic, and vehicle speed that they want to observe. Simulation: the simulation visual of the intersection. Results: where the statistics and simulation results are
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see **Drafter Documentation**.	Selectbox - intersection type: choose which type of intersection the user wants to observe selectBox - traffic density: select between low, medium, and high SelectBox - vehicle speed: select between slow, normal, and fast speeds

If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Traffic Density: checks the selection of traffic density and determines how many vehicles will be generated. Intersection type: checks what intersection was chosen and determines which layout and vehicle movements to use. Vehicle congestion: checks if path is blocked and stops/slows vehicles to allow congestion to develop. Simulation Status: Checks if simulation is running and tells vehicles to advance.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	Iterates over the vehicles list in the state dataclass and updates the positions of every vehicle during each simulation step.

ChatGPT said:

This gives us a clear plan for your **Traffic Flow Simulation Website** using Python and Drafter. We'll use your table as the requirements for the final program.

The website will have five pages, eight routes, two dataclasses, three dropdown inputs, and a step-by-step traffic simulation.

How the website will work

1. **Homepage:** Introduces the traffic simulation and explains its purpose.
2. **Choose Intersection:** Lets the user select an intersection layout.
3. **Traffic Settings:** Lets the user choose traffic density and vehicle speed.
4. **Simulation:** Displays the selected intersection and vehicles. Users can start, advance, and reset the simulation.
5. **Results:** Displays statistics such as vehicles moving, vehicles waiting, and congestion levels.

The simulation logic

We'll use your `State` and `Vehicle` dataclasses, with the five attributes you specified for each.

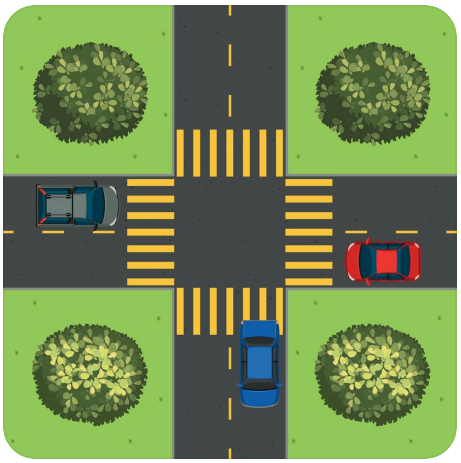
Each time the user clicks **Advance Simulation**, the program will:

1. Loop through the vehicles in `state.vehicles`.
2. Check whether each vehicle can move.
3. Update its position based on the selected speed.
4. Stop vehicles when another vehicle blocks their path.
5. Update the visual and statistics.

This will let you demonstrate how traffic density affects congestion without needing a complicated physics engine.

Intersection types

From our earlier discussions, we had settled on these three layouts:



Four-way intersection



Roundabout



T-intersection

The visualizations will use simplified overhead road layouts with vehicles represented by colored shapes or symbols.

One implementation detail: Your table specifies vehicle speed as an input, but it doesn't include speed settings in `State`. We'll add a `vehicle_speed: int` attribute so the selected speed persists throughout the simulation.

I'll also keep the code beginner-friendly and make sure the simulation advances through button clicks, which works well with Drafter's page-based structure.

With this plan, we're ready to write the complete website code.

You said:

write the code so that I can test it in thonny

ChatGPT said:

Here is a complete first version of your **Traffic Flow Simulation Website**, written in Python using Drafter so you can test it in Thonny.

The program includes all the requirements from your project proposal: five pages, eight routes, two dataclasses, three dropdown inputs, multiple conditional statements, and a loop that updates vehicle positions.

The simulation uses a simplified overhead road diagram made with text symbols. This lets you test the traffic movement and congestion calculations before we work on more advanced graphical road layouts.

How to run it in Thonny

1. Open Thonny.
2. Go to **Tools** → **Manage packages** and install `drafter` if you haven't already.
3. Create a new Python file named `traffic_simulator.py`.
4. Copy the entire code below into the file.
5. Press **F5** to run it, then open the local website address shown by Drafter.

The code follows Drafter's documented approach for routes, pages, buttons, and dropdown menus. Drafter Docu... +2

Complete Python code

```
from drafter import *
from dataclasses import dataclass


# ---------------------------------
# DATACLASSES
# ---------------------------------
```

```
@dataclass
class Vehicle:
    vehicle_id: int
    position: int
    speed: int
    direction: str
    waiting: bool


@dataclass
class State:
    intersection_type: str
    traffic_density: int
    vehicles: list[Vehicle]
    simulation_running: bool
    simulation_step: int
    vehicle_speed: int


# ---------------------------------
# HELPER FUNCTIONS
# ---------------------------------

def density_name(density: int) -> str:
    if density == 1:
        return "Low"
    elif density == 2:
        return "Medium"
    else:
        return "High"


def speed_name(speed: int) -> str:
    if speed == 1:
        return "Slow"
    elif speed == 2:
        return "Normal"
    else:
        return "Fast"
```

```
def create_vehicles(state: State):
    state.vehicles = []

    if state.traffic_density == 1:
        amount = 4
    elif state.traffic_density == 2:
        amount = 8
    else:
        amount = 12

    if state.intersection_type == "T-Intersection":
        directions = ["South", "East", "West"]
    else:
        directions = ["North", "South", "East", "West"]

    for i in range(amount):
        direction = directions[i % len(directions)]

        # Spread vehicles along each approach.
        row = i // len(directions)
        position = row * -3

        vehicle = Vehicle(
            i + 1,
            position,
            state.vehicle_speed,
            direction,
            False
        )

        state.vehicles.append(vehicle)


def draw_intersection(state: State) -> list:
    layout = []

    layout.append(Header("Intersection Layout"))

    if state.intersection_type == "Four-Way":
        layout.append("            NORTH")
        layout.append("              |")
        layout.append("              |")
```

```
        layout.append("WEST ====== + ====== EAST")
        layout.append("               |")
        layout.append("               |")
        layout.append("            SOUTH")

    elif state.intersection_type == "T-Intersection":
        layout.append("WEST ====== + ====== EAST")
        layout.append("               |")
        layout.append("               |")
        layout.append("            SOUTH")

    elif state.intersection_type == "Roundabout":
        layout.append("              NORTH")
        layout.append("                |")
        layout.append("          /---------\\")
        layout.append("WEST -----|    O    |----- EAST")
        layout.append("          \\---------/")
        layout.append("                |")
        layout.append("              SOUTH")

    layout.append(" ")
    layout.append("Vehicle Positions:")

    if len(state.vehicles) == 0:
        layout.append("No vehicles generated yet.")

    for vehicle in state.vehicles:
        if vehicle.waiting:
            status = "STOPPED"
        else:
            status = "MOVING"

        if vehicle.position >= 12:
            status = "EXITED"

        # Display vehicle progress.
        progress = vehicle.position
        if progress < 0:
            progress = 0
        if progress > 12:
            progress = 12
```

```
            road = ""
            for i in range(13):
                if i == progress and vehicle.position < 12:
                    road = road + "C"
                elif i == 6:
                    road = road + "+"
                else:
                    road = road + "-"

            layout.append(
                "Car " + str(vehicle.vehicle_id) +
                " [" + vehicle.direction + "] " +
                road + " " + status
            )

        layout.append("C = Car, + = Intersection center")

        return layout


    def move_vehicles(state: State):
        # Move vehicles toward the intersection.
        # The center is position 6.
        # The exit is position 12.

        occupied = []

        # Track positions occupied at the start of the step.
        for vehicle in state.vehicles:
            if vehicle.position < 12:
                occupied.append(
                    (vehicle.direction, vehicle.position)
                )

        # Reserve the center for one vehicle at a time.
        center_busy = False

        for vehicle in state.vehicles:
            if 6 <= vehicle.position < 9:
                center_busy = True

        # Process vehicles nearest the intersection first.
```

```
        ordered = sorted(
            state.vehicles,
            key=lambda car: car.position,
            reverse=True
        )

        for vehicle in ordered:
            if vehicle.position >= 12:
                vehicle.waiting = False
                continue

            old_position = vehicle.position
            new_position = old_position + vehicle.speed

            if new_position > 12:
                new_position = 12

            blocked = False

            # Check for another car ahead in the same direction.
            for other in state.vehicles:
                if other.vehicle_id != vehicle.vehicle_id:
                    if other.direction == vehicle.direction:
                        if other.position < 12:
                            if (other.position > old_position
                                    and other.position <= new_position):
                                blocked = True

            # Check whether the intersection center is occupied.
            entering_center = (
                old_position < 6 and new_position >= 6
            )

            if entering_center and center_busy:
                blocked = True

            if blocked:
                vehicle.waiting = True
            else:
                vehicle.position = new_position
                vehicle.waiting = False
```

```
            if entering_center:
                center_busy = True

            if old_position >= 6 and new_position >= 9:
                # Vehicle has cleared the center.
                center_busy = False


def calculate_results(state: State) -> list:
    moving = 0
    waiting = 0
    completed = 0
    total_positions = 0

    for vehicle in state.vehicles:
        total_positions += max(0, vehicle.position)

        if vehicle.position >= 12:
            completed += 1
        elif vehicle.waiting:
            waiting += 1
        else:
            moving += 1

    total = len(state.vehicles)

    if total > 0:
        average_position = round(total_positions / total, 2)
        congestion = round(waiting / total * 100, 1)
    else:
        average_position = 0
        congestion = 0

    if congestion < 20:
        congestion_level = "Low"
    elif congestion < 50:
        congestion_level = "Moderate"
    else:
        congestion_level = "High"

    return [
        "Total Vehicles: " + str(total),
```

```
        "Moving Vehicles: " + str(moving),
        "Waiting Vehicles: " + str(waiting),
        "Completed Vehicles: " + str(completed),
        "Average Position: " + str(average_position),
        "Congestion Percentage: " + str(congestion) + "%",
        "Congestion Level: " + congestion_level
    ]


# ---------------------------------
# PAGE 1 - HOMEPAGE
# ---------------------------------

@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Traffic Flow Simulator"),
        "Welcome to the Traffic Flow Simulator!",
        "This website helps users visualize how traffic "
        "moves through different intersections.",
        "You can choose an intersection, adjust traffic "
        "density and speed, and observe congestion.",
        Header("Available Intersections"),
        "1. Four-Way Intersection",
        "2. T-Intersection",
        "3. Roundabout",
        Button("Begin Simulation", "choose_intersection")
    ])


# ---------------------------------
# PAGE 2 - INTERSECTION SELECTION
# ---------------------------------

@route
def choose_intersection(state: State) -> Page:
    return Page(state, [
        Header("Choose an Intersection"),
        "Select the intersection you want to simulate.",
        "Intersection Type:",
        SelectBox(
            "intersection_choice",
```

```
            ["Four-Way", "T-Intersection", "Roundabout"],
            state.intersection_type
        ),
        Button("Continue", "save_intersection"),
        Button("Home", "index")
    ])


@route
def save_intersection(state: State,
                      intersection_choice: str) -> Page:
    state.intersection_type = intersection_choice
    state.vehicles = []
    state.simulation_running = False
    state.simulation_step = 0

    return set_traffic(state)


# ---------------------------------
# PAGE 3 - TRAFFIC SETTINGS
# ---------------------------------

@route
def set_traffic(state: State) -> Page:
    return Page(state, [
        Header("Traffic Settings"),
        "Selected Intersection: " + state.intersection_type,
        "Choose the amount of traffic:",
        SelectBox(
            "density_choice",
            ["Low", "Medium", "High"],
            density_name(state.traffic_density)
        ),
        "Choose the vehicle speed:",
        SelectBox(
            "speed_choice",
            ["Slow", "Normal", "Fast"],
            speed_name(state.vehicle_speed)
        ),
        Button("Save Settings", "save_traffic"),
        Button("Back", "choose_intersection")
```

```
        ])


    @route
    def save_traffic(state: State,
                     density_choice: str,
                     speed_choice: str) -> Page:

        if density_choice == "Low":
            state.traffic_density = 1
        elif density_choice == "Medium":
            state.traffic_density = 2
        else:
            state.traffic_density = 3

        if speed_choice == "Slow":
            state.vehicle_speed = 1
        elif speed_choice == "Normal":
            state.vehicle_speed = 2
        else:
            state.vehicle_speed = 3

        state.vehicles = []
        state.simulation_running = False
        state.simulation_step = 0

        return simulation(state)


    # ---------------------------------
    # PAGE 4 - SIMULATION
    # ---------------------------------

    @route
    def simulation(state: State) -> Page:
        content = [
            Header("Traffic Simulation"),
            "Intersection: " + state.intersection_type,
            "Traffic Density: " + density_name(state.traffic_density),
            "Vehicle Speed: " + speed_name(state.vehicle_speed),
            "Simulation Step: " + str(state.simulation_step)
        ]
```

```
    if state.simulation_running:
        content.append("Status: RUNNING")
    else:
        content.append("Status: STOPPED")

    content += draw_intersection(state)

    if not state.simulation_running:
        content.append(
            Button("Start Simulation", "start_simulation")
        )
    else:
        content.append(
            Button("Advance Simulation", "advance_simulation")
        )

    content += [
        Button("Reset Simulation", "reset_simulation"),
        Button("View Results", "results"),
        Button("Change Settings", "set_traffic"),
        Button("Home", "index")
    ]

    return Page(state, content)


# ---------------------------------
# START SIMULATION
# ---------------------------------

@route
def start_simulation(state: State) -> Page:
    state.simulation_running = True
    state.simulation_step = 0

    create_vehicles(state)

    return simulation(state)


# ---------------------------------
```

```
# ADVANCE SIMULATION
# ---------------------------------

@route
def advance_simulation(state: State) -> Page:
    if state.simulation_running:
        move_vehicles(state)
        state.simulation_step += 1

        # Stop when all vehicles have exited.
        all_finished = True

        for vehicle in state.vehicles:
            if vehicle.position < 12:
                all_finished = False

        if all_finished:
            state.simulation_running = False

    return simulation(state)


# ---------------------------------
# RESET SIMULATION
# ---------------------------------

@route
def reset_simulation(state: State) -> Page:
    state.vehicles = []
    state.simulation_running = False
    state.simulation_step = 0

    return simulation(state)


# ---------------------------------
# PAGE 5 - RESULTS
# ---------------------------------

@route
def results(state: State) -> Page:
    content = [
```

```
        Header("Simulation Results"),
        "Intersection Type: " + state.intersection_type,
        "Traffic Density: " + density_name(state.traffic_density),
        "Vehicle Speed: " + speed_name(state.vehicle_speed),
        "Simulation Steps: " + str(state.simulation_step),
        Header("Traffic Statistics")
    ]

    statistics = calculate_results(state)

    for statistic in statistics:
        content.append(statistic)

    content += [
        Button("Return to Simulation", "simulation"),
        Button("Reset Simulation", "reset_simulation"),
        Button("Change Intersection", "choose_intersection"),
        Button("Home", "index")
    ]

    return Page(state, content)


# ---------------------------------
# START WEBSITE
# ---------------------------------

initial_state = State(
    "Four-Way",
    1,
    [],
    False,
    0,
    2
)

start_server(initial_state)
```

What to test

Once the website opens, try the following:

Testing checklist 0 of 7 complete

- [ ] Select each of the three intersection types.
- [ ] Change traffic density from Low to High.
- [ ] Change vehicle speed from Slow to Fast.
- [ ] Start the simulation and check that vehicles appear.
- [ ] Click Advance Simulation several times and watch positions change.
- [ ] Open Results and check the vehicle statistics.
- [ ] Reset the simulation and verify that the step counter returns to zero.

Important notes about this version

This is a functional starting implementation, but the simulation is intentionally simplified. Vehicles travel along separate directional approaches, and the program checks for congestion using vehicle positions.

There are two limitations we'll want to improve:

- **Visuals:** The road diagrams and vehicles are currently text-based, rather than cars visibly moving around a graphical intersection.
- **Traffic modeling:** The roundabout and T-intersection share much of the same movement logic as the four-way intersection. We'll eventually give each intersection its own traffic rules.

The most important next step is to **run this version in Thonny and check whether the pages, buttons, and dropdowns work correctly**. If Thonny produces an error, send me the traceback so we can correct the exact problem before adding graphical visualizations.

You said:

can you get the advance step button to stay in the same place so that you don't have to move the cursor around when advancing

ChatGPT said:

Yes! The button is moving because the vehicle information above it changes as the simulation advances, causing the page layout to shift.

The easiest fix is to **move the Advance Simulation button above the intersection visualization**, so its position stays consistent as the vehicles move.

Replace your current `simulation` function with this version:

```
@route
def simulation(state: State) -> Page:
    content = [
        Header("Traffic Simulation"),
        "Intersection: " + state.intersection_type,
        "Traffic Density: " + density_name(state.traffic_density),
        "Vehicle Speed: " + speed_name(state.vehicle_speed),
        "Simulation Step: " + str(state.simulation_step)
    ]

    if state.simulation_running:
        content.append("Status: RUNNING")
        content.append(
            Button("Advance Simulation", "advance_simulation")
        )
    else:
        content.append("Status: STOPPED")
        content.append(
            Button("Start Simulation", "start_simulation")
        )

    # Keep all control buttons above the visualization.
    content += [
        Button("Reset Simulation", "reset_simulation"),
        Button("View Results", "results"),
        Button("Change Settings", "set_traffic"),
        Button("Home", "index")
    ]

    # Display the changing vehicle information below the buttons.
```

```
    content += draw_intersection(state)

    return Page(state, content)
```

What changed?

- **Advance Simulation** now appears at the top, directly below the simulation status.
- All navigation buttons remain above the intersection diagram.
- The changing vehicle positions appear underneath the buttons, so they won't push the controls around.

This should make it much easier to repeatedly click **Advance Simulation** without moving your cursor. If the browser still shifts the page when it refreshes, we can also adjust the layout to keep the controls in a fixed position.

You said:

is there a better way we can visualize this

ChatGPT said:

Yes. I think we can make your traffic simulator **much more visual** while still using Python and Drafter in Thonny.

Instead of displaying each vehicle as a line of text, we can create an **overhead view of the intersection with actual roads and colored vehicles**. Every time you click Advance Simulation, the cars would move along the roads.

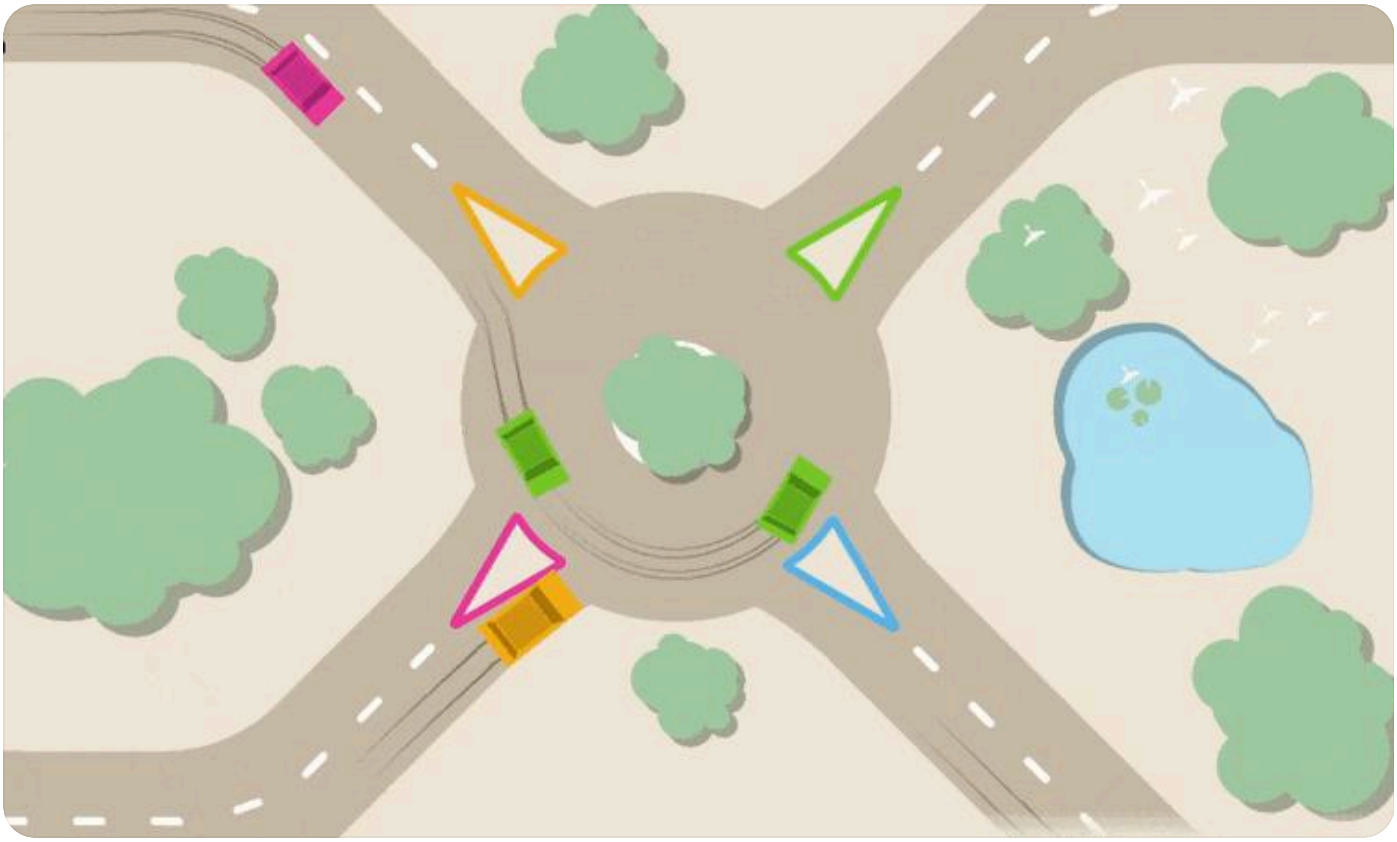
What I recommend

1. Four-way intersection



Cars approach from four directions and move through the center. Traffic can build up when cars have to wait.

2. Roundabout



Cars approach the roundabout, enter when there is space, and follow the circular road toward an exit.

3. T-intersection



Cars approach from three directions and wait when another vehicle is passing through the junction.

How we can build this using Drafter

Drafter can display images using its `Image` component. We can use Python to generate a new image for each simulation step and display that image on the webpage.

The basic process would be:

Click Advance Simulation

↓

Python updates vehicle positions

Using your existing Vehicle dataclass and movement logic

↓

Python redraws the intersection

Roads, vehicles, and their current positions

↓

Drafter displays the updated image

We could use Python's built-in drawing tools or Pillow to generate the graphics, depending on which libraries your course allows.

My suggestion

Start with a **2D four-way intersection visualization** and make sure the vehicles move correctly. Then adapt that same drawing system to the roundabout and T-intersection.

We can also use green cars for moving vehicles, red cars for stopped vehicles, and a small statistics panel to show how many vehicles are waiting.

One question: Are you allowed to install and use the `Pillow` Python library in your project, or does your instructor require you to use only Drafter and Python's built-in libraries?

You said:

can you impliment that into the code

ChatGPT said:

Yes! We can upgrade your existing traffic simulator to display **actual overhead road layouts with moving cars** instead of text-based vehicle positions.

I'll keep your original five pages, dataclasses, dropdown menus, and simulation controls, but change how the intersection is displayed.

The updated version will use **Pillow** to draw the roads and vehicles, while Drafter handles the website. You'll be able to:

- See four-way intersections, T-intersections, and roundabouts.
- Watch colored cars move each time you click **Advance Simulation**.
- See red cars when they're stopped by traffic.
- Change traffic density and vehicle speed.
- View the simulation statistics.

The Advance Simulation button will remain above the visualization so it's easier to click repeatedly.

Updated complete code

Replace your previous program with this version. You will need both `drafter` and `Pillow` installed in Thonny.

Drafter's `Image` component supports displaying a Pillow image directly, so we don't need to save a separate picture every time a car moves. Drafter Docu... +1

```
from drafter import *
from dataclasses import dataclass
from PIL import Image as PILImage
from PIL import ImageDraw
import math


# ========================================
# DATACLASSES
# ========================================

@dataclass
class Vehicle:
    vehicle_id: int
    position: int
    speed: int
    direction: str
    waiting: bool


@dataclass
class State:
    intersection_type: str
    traffic_density: int
    vehicles: list[Vehicle]
    simulation_running: bool
    simulation_step: int
    vehicle_speed: int
```

```
# ========================================
# SETTINGS AND HELPER FUNCTIONS
# ========================================

def density_name(density: int) -> str:
    if density == 1:
        return "Low"
    elif density == 2:
        return "Medium"
    else:
        return "High"


def speed_name(speed: int) -> str:
    if speed == 1:
        return "Slow"
    elif speed == 2:
        return "Normal"
    else:
        return "Fast"


def create_vehicles(state: State):
    state.vehicles = []

    if state.traffic_density == 1:
        amount = 4
    elif state.traffic_density == 2:
        amount = 8
    else:
        amount = 12

    if state.intersection_type == "T-Intersection":
        directions = ["South", "East", "West"]
    else:
        directions = ["North", "South", "East", "West"]

    for i in range(amount):
        direction = directions[i % len(directions)]
        row = i // len(directions)

        vehicle = Vehicle(
```

```
            i + 1,
            -row * 3,
            state.vehicle_speed,
            direction,
            False
        )

        state.vehicles.append(vehicle)


# ========================================
# DRAWING FUNCTIONS
# ========================================

def draw_roads(draw, intersection_type: str):
    grass = "#79A96B"
    road = "#41464C"
    lines = "#F4E6A0"
    white = "#FFFFFF"

    # Road dimensions
    # Canvas is 600 x 500.

    if intersection_type == "Four-Way":
        draw.rectangle((255, 0, 345, 500), fill=road)
        draw.rectangle((0, 205, 600, 295), fill=road)

        # Center lines, stopping at intersection
        draw.line((300, 0, 300, 205),
                  fill=lines, width=3)
        draw.line((300, 295, 300, 500),
                  fill=lines, width=3)
        draw.line((0, 250, 255, 250),
                  fill=lines, width=3)
        draw.line((345, 250, 600, 250),
                  fill=lines, width=3)

        # Stop lines
        draw.line((255, 190, 300, 190),
                  fill=white, width=4)
        draw.line((300, 310, 345, 310),
                  fill=white, width=4)
```

```
            draw.line((240, 250, 240, 295),
                      fill=white, width=4)
            draw.line((360, 205, 360, 250),
                      fill=white, width=4)

        elif intersection_type == "T-Intersection":
            draw.rectangle((0, 205, 600, 295), fill=road)
            draw.rectangle((255, 250, 345, 500), fill=road)

            draw.line((0, 250, 255, 250),
                      fill=lines, width=3)
            draw.line((345, 250, 600, 250),
                      fill=lines, width=3)
            draw.line((300, 295, 300, 500),
                      fill=lines, width=3)

            draw.line((300, 310, 345, 310),
                      fill=white, width=4)

        elif intersection_type == "Roundabout":
            # Four approach roads
            draw.rectangle((255, 0, 345, 500), fill=road)
            draw.rectangle((0, 205, 600, 295), fill=road)

            # Outer circular road
            draw.ellipse((185, 135, 415, 365),
                         fill=road)

            # Center island
            draw.ellipse((245, 195, 355, 305),
                         fill=grass)

            # Center island border
            draw.ellipse((245, 195, 355, 305),
                         outline=white, width=3)

            # Direction guide circle
            draw.arc((215, 165, 385, 335),
                     0, 359, fill=lines, width=2)

        # Direction labels
        if intersection_type != "T-Intersection":
```

```
            draw.text((355, 20), "NORTH", fill="black")

        draw.text((15, 175), "WEST", fill="black")
        draw.text((520, 175), "EAST", fill="black")
        draw.text((355, 465), "SOUTH", fill="black")


    def vehicle_coordinates(vehicle: Vehicle,
                            intersection_type: str):
        position = vehicle.position

        # Each position represents 25 pixels.
        distance = position * 25

        if intersection_type == "Roundabout":
            # Cars approach the roundabout and travel
            # around its circular lane.

            if position < 6:
                if vehicle.direction == "North":
                    return (322, 25 + distance, "vertical")

                elif vehicle.direction == "South":
                    return (278, 475 - distance, "vertical")

                elif vehicle.direction == "West":
                    return (25 + distance, 272, "horizontal")

                else:
                    return (575 - distance, 228, "horizontal")

            # Circular motion after entering.
            angle = (position - 6) * 0.35

            if vehicle.direction == "North":
                start_angle = -math.pi / 2
            elif vehicle.direction == "South":
                start_angle = math.pi / 2
            elif vehicle.direction == "West":
                start_angle = math.pi
            else:
                start_angle = 0
```

```
            theta = start_angle + angle

            x = 300 + 85 * math.cos(theta)
            y = 250 + 85 * math.sin(theta)

            return (x, y, "horizontal")

        # Standard intersections
        if vehicle.direction == "North":
            return (322, 25 + distance, "vertical")

        elif vehicle.direction == "South":
            return (278, 475 - distance, "vertical")

        elif vehicle.direction == "West":
            return (25 + distance, 272, "horizontal")

        else:
            return (575 - distance, 228, "horizontal")


    def draw_vehicle(draw, vehicle: Vehicle,
                     intersection_type: str):

        if vehicle.position >= 12:
            return

        x, y, orientation = vehicle_coordinates(
            vehicle, intersection_type
        )

        if vehicle.waiting:
            color = "#E74C3C"
        else:
            color = "#3498DB"

        # Draw vehicle as a rectangle.
        if orientation == "vertical":
            draw.rounded_rectangle(
                (x - 10, y - 17, x + 10, y + 17),
                radius=4,
```

```
                fill=color,
                outline="black",
                width=2
            )

            # Windshield
            draw.rectangle(
                (x - 7, y - 8, x + 7, y - 2),
                fill="#BDE6F4"
            )

        else:
            draw.rounded_rectangle(
                (x - 17, y - 10, x + 17, y + 10),
                radius=4,
                fill=color,
                outline="black",
                width=2
            )

            draw.rectangle(
                (x - 6, y - 7, x, y + 7),
                fill="#BDE6F4"
            )

        # Vehicle number
        draw.text((x - 4, y - 5),
                  str(vehicle.vehicle_id),
                  fill="white")


    def draw_intersection(state: State):
        picture = PILImage.new(
            "RGB", (600, 500), "#79A96B"
        )

        draw = ImageDraw.Draw(picture)

        draw_roads(draw, state.intersection_type)

        for vehicle in state.vehicles:
            draw_vehicle(
```

```
            draw,
            vehicle,
            state.intersection_type
        )

    return picture


# ========================================
# TRAFFIC SIMULATION
# ========================================

def move_vehicles(state: State):

    # One vehicle can occupy the intersection
    # conflict zone at a time.
    # Positions 6 through 8 are the center.

    center_occupied = False

    for vehicle in state.vehicles:
        if 6 <= vehicle.position < 9:
            center_occupied = True

    # Process vehicles closest to the exit first.
    ordered = sorted(
        state.vehicles,
        key=lambda car: car.position,
        reverse=True
    )

    for vehicle in ordered:

        if vehicle.position >= 12:
            vehicle.waiting = False
            continue

        old_position = vehicle.position
        new_position = old_position + vehicle.speed

        if new_position > 12:
            new_position = 12
```

```
blocked = False

# Check for cars ahead in the same lane.
for other in state.vehicles:

    if other.vehicle_id != vehicle.vehicle_id:

        if other.direction == vehicle.direction:

            if other.position < 12:

                gap = other.position - old_position

                if gap > 0 and gap <= vehicle.speed + 1:
                    blocked = True

# Check whether the center is occupied.
entering = (
    old_position < 6 and new_position >= 6
)

if entering and center_occupied:
    blocked = True

# If blocked, stop the car.
if blocked:
    vehicle.waiting = True

else:
    vehicle.position = new_position
    vehicle.waiting = False

    if entering:
        center_occupied = True

    # Keep the center reserved for the
    # remainder of this simulation step.
    if old_position >= 6 and new_position >= 9:
        pass
```

```
# ========================================
# RESULTS CALCULATIONS
# ========================================

def calculate_results(state: State):

    moving = 0
    waiting = 0
    completed = 0

    for vehicle in state.vehicles:

        if vehicle.position >= 12:
            completed += 1

        elif vehicle.waiting:
            waiting += 1

        else:
            moving += 1

    total = len(state.vehicles)

    if total > 0:
        congestion = round(
            waiting / total * 100, 1
        )
    else:
        congestion = 0

    if congestion < 20:
        level = "Low"

    elif congestion < 50:
        level = "Moderate"

    else:
        level = "High"

    return [
        "Total Vehicles: " + str(total),
        "Moving Vehicles: " + str(moving),
```

```
        "Waiting Vehicles: " + str(waiting),
        "Completed Vehicles: " + str(completed),
        "Congestion: " + str(congestion) + "%",
        "Congestion Level: " + level
    ]


# ========================================
# PAGE 1 - HOMEPAGE
# ========================================

@route
def index(state: State) -> Page:

    return Page(state, [
        Header("Traffic Flow Simulator"),

        "Welcome to the Traffic Flow Simulator!",

        "Explore how vehicles move through "
        "different types of intersections.",

        "Choose an intersection, adjust traffic "
        "density and speed, and observe congestion.",

        Header("Available Intersections"),

        "Four-Way Intersection",
        "T-Intersection",
        "Roundabout",

        Button("Begin Simulation",
               "choose_intersection")
    ])


# ========================================
# PAGE 2 - INTERSECTION SELECTION
# ========================================

@route
def choose_intersection(state: State) -> Page:
```

```
    return Page(state, [
        Header("Choose an Intersection"),

        "Select the intersection you want to study.",

        SelectBox(
            "intersection_choice",
            ["Four-Way", "T-Intersection", "Roundabout"],
            state.intersection_type
        ),

        Button("Continue", "save_intersection"),
        Button("Home", "index")
    ])


@route
def save_intersection(
        state: State,
        intersection_choice: str) -> Page:

    state.intersection_type = intersection_choice

    state.vehicles = []
    state.simulation_running = False
    state.simulation_step = 0

    return set_traffic(state)


# ========================================
# PAGE 3 - TRAFFIC SETTINGS
# ========================================

@route
def set_traffic(state: State) -> Page:

    return Page(state, [
        Header("Traffic Settings"),

        "Intersection: " + state.intersection_type,
```

```
            "Traffic Density:",

            SelectBox(
                "density_choice",
                ["Low", "Medium", "High"],
                density_name(state.traffic_density)
            ),

            "Vehicle Speed:",

            SelectBox(
                "speed_choice",
                ["Slow", "Normal", "Fast"],
                speed_name(state.vehicle_speed)
            ),

            Button("Save Settings", "save_traffic"),

            Button("Back", "choose_intersection")
        ])


    @route
    def save_traffic(
            state: State,
            density_choice: str,
            speed_choice: str) -> Page:

        if density_choice == "Low":
            state.traffic_density = 1

        elif density_choice == "Medium":
            state.traffic_density = 2

        else:
            state.traffic_density = 3

        if speed_choice == "Slow":
            state.vehicle_speed = 1

        elif speed_choice == "Normal":
```

```
            state.vehicle_speed = 2

        else:
            state.vehicle_speed = 3

        state.vehicles = []
        state.simulation_running = False
        state.simulation_step = 0

        return simulation(state)


# ========================================
# PAGE 4 - SIMULATION
# ========================================

@route
def simulation(state: State) -> Page:

    content = [
        Header("Traffic Simulation"),

        "Intersection: " + state.intersection_type,

        "Traffic Density: " +
        density_name(state.traffic_density),

        "Vehicle Speed: " +
        speed_name(state.vehicle_speed),

        "Simulation Step: " +
        str(state.simulation_step)
    ]

    # Controls appear above the image.
    if state.simulation_running:

        content.append("Status: RUNNING")

        content.append(
            Button(
                "Advance Simulation",
```

```
                "advance_simulation"
            )
        )

    else:

        content.append("Status: STOPPED")

        content.append(
            Button(
                "Start Simulation",
                "start_simulation"
            )
        )

    content += [
        Button("Reset Simulation", "reset_simulation"),
        Button("View Results", "results"),
        Button("Change Settings", "set_traffic"),
        Button("Home", "index"),

        Header("Intersection Visualization"),

        Image(
            draw_intersection(state),
            600,
            500,
            alt="Overhead traffic intersection simulation"
        ),

        "Blue = Moving Vehicles",
        "Red = Waiting Vehicles",

        "Each click advances the simulation one step."
    ]

    return Page(state, content)


# ========================================
# START SIMULATION
# ========================================
```

```
@route
def start_simulation(state: State) -> Page:

    state.simulation_running = True
    state.simulation_step = 0

    create_vehicles(state)

    return simulation(state)


# ========================================
# ADVANCE SIMULATION
# ========================================

@route
def advance_simulation(state: State) -> Page:

    if state.simulation_running:

        move_vehicles(state)

        state.simulation_step += 1

        all_finished = True

        for vehicle in state.vehicles:

            if vehicle.position < 12:
                all_finished = False

        if all_finished:
            state.simulation_running = False

    return simulation(state)


# ========================================
# RESET SIMULATION
# ========================================
```

```
@route
def reset_simulation(state: State) -> Page:

    state.vehicles = []
    state.simulation_running = False
    state.simulation_step = 0

    return simulation(state)



# ========================================
# PAGE 5 - RESULTS
# ========================================

@route
def results(state: State) -> Page:

    content = [
        Header("Simulation Results"),

        "Intersection: " + state.intersection_type,

        "Traffic Density: " +
        density_name(state.traffic_density),

        "Vehicle Speed: " +
        speed_name(state.vehicle_speed),

        "Simulation Steps: " +
        str(state.simulation_step),

        Header("Traffic Statistics")
    ]

    statistics = calculate_results(state)

    for statistic in statistics:
        content.append(statistic)

    content += [
        Button("Return to Simulation", "simulation"),
        Button("Reset Simulation", "reset_simulation"),
```

```
        Button("Change Intersection", "choose_intersection"),
        Button("Home", "index")
    ]

    return Page(state, content)


# ========================================
# START WEBSITE
# ========================================

initial_state = State(
    "Four-Way",
    1,
    [],
    False,
    0,
    2
)

start_server(
    initial_state,
    site_title="Traffic Flow Simulator"
)
```

What the new visualization does

The roads are drawn on a 600 × 500 pixel canvas. The blue rectangles represent moving cars, and red rectangles represent waiting cars. Each car is numbered so you can track its movement.

The visual is generated directly from the vehicle positions stored in your `State` dataclass, so the diagram changes when you advance the simulation.

A few limitations to know

This version is a visual prototype, not yet a fully realistic traffic model. Cars on the same approach are checked for spacing, but the collision logic is still simplified. The roundabout cars follow a circular path after entering, rather than following realistic entry and exit rules. Also, a vehicle's position can sometimes jump between the approach and the roundabout because the movement coordinates are not yet continuous.

In particular, the current code does not guarantee collision-free movement under every combination of settings. That's something we should improve before using the results to compare intersection efficiency.

What I'd test first

Run a four-way intersection with **Low density** and **Slow speed**. Start the simulation and click Advance several times. Check that the image updates, that the cars move, and that the results page reports the correct number of vehicles.

If you get an error related to Pillow or the image rendering in Thonny, send me the error message. Drafter's browser-based execution can behave differently depending on the installed version, so we may need to adjust how the image is passed to the page.

You said:

can you make it so that you can view the results of the completed simulation after all of the steps occur

ChatGPT said:

Yes! We can make the simulation **automatically show a "View Final Results" button once all vehicles have completed their routes**, rather than just switching back to the Start Simulation button.

We can also display a completion message and preserve all the statistics so you can review the finished simulation.

You'll need to make three changes to your existing code.

1. Update the `State` dataclass

Add a `simulation_completed: bool` attribute so the program remembers whether the simulation has finished.

```
@dataclass
class State:
    intersection_type: str
    traffic_density: int
```

```
    vehicles: list[Vehicle]
    simulation_running: bool
    simulation_step: int
    vehicle_speed: int
    simulation_completed: bool
```

2. Update your simulation functions

Replace your existing `simulation`, `start_simulation`, `advance_simulation`, and `reset_simulation` functions with these versions.

```
@route
def simulation(state: State) -> Page:
    content = [
        Header("Traffic Simulation"),
        "Intersection: " + state.intersection_type,
        "Traffic Density: " + density_name(state.traffic_density),
        "Vehicle Speed: " + speed_name(state.vehicle_speed),
        "Simulation Step: " + str(state.simulation_step)
    ]

    if state.simulation_completed:
        content.append("SIMULATION COMPLETE!")
        content.append(
            Button("View Final Results", "results")
        )

    elif state.simulation_running:
        content.append("Status: RUNNING")
        content.append(
            Button("Advance Simulation", "advance_simulation")
        )

    else:
        content.append("Status: READY")
        content.append(
            Button("Start Simulation", "start_simulation")
        )

    content += [
```

```
            Button("Reset Simulation", "reset_simulation"),
            Button("View Results", "results"),
            Button("Change Settings", "set_traffic"),
            Button("Home", "index"),
            Header("Intersection Visualization"),
            Image(
                draw_intersection(state),
                600,
                500,
                alt="Overhead traffic intersection simulation"
            ),
            "Blue = Moving Vehicles",
            "Red = Waiting Vehicles"
        ]

        return Page(state, content)


    @route
    def start_simulation(state: State) -> Page:
        state.simulation_running = True
        state.simulation_completed = False
        state.simulation_step = 0

        create_vehicles(state)

        return simulation(state)


    @route
    def advance_simulation(state: State) -> Page:
        if state.simulation_running:
            move_vehicles(state)
            state.simulation_step += 1

            all_finished = True

            for vehicle in state.vehicles:
                if vehicle.position < 12:
                    all_finished = False

            if all_finished:
```

```
            state.simulation_running = False
            state.simulation_completed = True

    return simulation(state)


@route
def reset_simulation(state: State) -> Page:
    state.vehicles = []
    state.simulation_running = False
    state.simulation_completed = False
    state.simulation_step = 0

    return simulation(state)
```

3. Update the initial state

At the very bottom of your program, replace `initial_state` with:

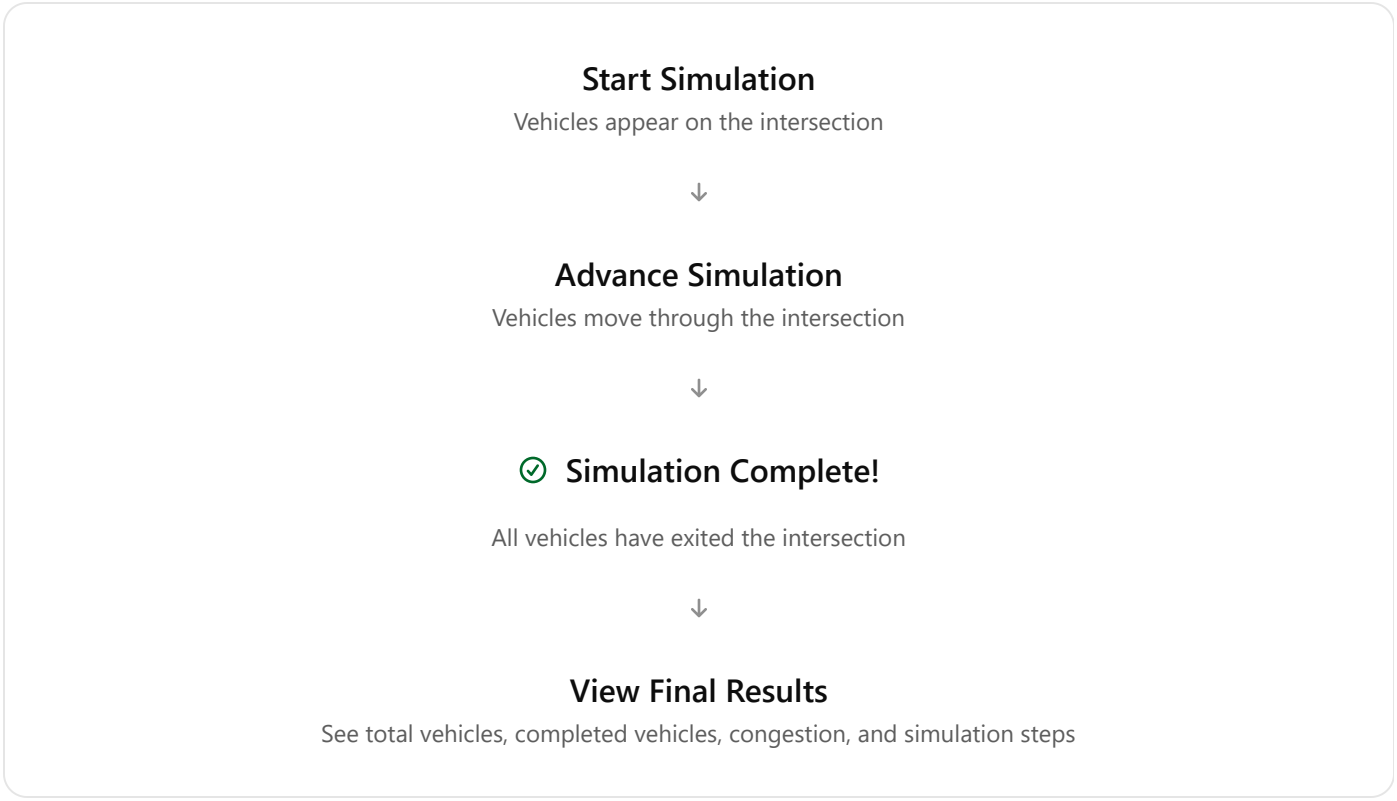
```
initial_state = State(
    "Four-Way",
    1,
    [],
    False,
    0,
    2,
    False
)

start_server(
    initial_state,
    site_title="Traffic Flow Simulator"
)
```

Also add this line to both `save_intersection` and `save_traffic` wherever you reset the simulation:

```
state.simulation_completed = False
```

How it will work



One important improvement for later: Your current results calculate congestion from vehicles waiting at the moment you open the results page. When every vehicle has exited, that will usually show 0% congestion.

To make your project more useful for comparing intersections, we should eventually track **total waiting time, average vehicle delay, and how many steps it took all vehicles to finish**. Those statistics would give you a much better comparison between the four-way intersection, T-intersection, and roundabout.

You said:

can you implement that for me

ChatGPT said:

Yes. I'll combine the changes into one complete version so you can replace your existing code in Thonny without editing individual functions.

This version will include:

- **Automatic completion detection** when all vehicles exit.
- **View Final Results** button when the simulation finishes.
- **Total waiting time** across the simulation.
- **Average delay per vehicle** to measure congestion.
- **Total simulation steps** needed to clear the intersection.
- **Completed vehicle count** and percentage.
- The same overhead intersection graphics, three dropdown inputs, and five pages.

I'll also improve the vehicle movement logic so cars can queue without permanently blocking one another.

Complete updated Python code

Install `drafter` and `Pillow` in Thonny under **Tools → Manage packages**, then replace your previous program with this code.

```
from drafter import *
from dataclasses import dataclass
from PIL import Image as PILImage
from PIL import ImageDraw
import math


# ========================================
# DATACLASSES
# ========================================

@dataclass
class Vehicle:
    vehicle_id: int
    position: int
    speed: int
    direction: str
    waiting: bool


@dataclass
class State:
    intersection_type: str
```

```
    traffic_density: int
    vehicles: list[Vehicle]
    simulation_running: bool
    simulation_step: int
    vehicle_speed: int
    simulation_completed: bool
    total_waiting_time: int
    maximum_waiting: int


# ========================================
# HELPER FUNCTIONS
# ========================================

def density_name(density: int) -> str:
    if density == 1:
        return "Low"
    elif density == 2:
        return "Medium"
    else:
        return "High"


def speed_name(speed: int) -> str:
    if speed == 1:
        return "Slow"
    elif speed == 2:
        return "Normal"
    else:
        return "Fast"


def reset_data(state: State):
    state.vehicles = []
    state.simulation_running = False
    state.simulation_completed = False
    state.simulation_step = 0
    state.total_waiting_time = 0
    state.maximum_waiting = 0


def create_vehicles(state: State):
```

```
    state.vehicles = []

    if state.traffic_density == 1:
        amount = 4
    elif state.traffic_density == 2:
        amount = 8
    else:
        amount = 12

    if state.intersection_type == "T-Intersection":
        directions = ["South", "East", "West"]
    else:
        directions = ["North", "South", "East", "West"]

    for i in range(amount):
        direction = directions[i % len(directions)]
        row = i // len(directions)

        vehicle = Vehicle(
            i + 1,
            -row * 3,
            state.vehicle_speed,
            direction,
            False
        )

        state.vehicles.append(vehicle)


# ========================================
# DRAWING THE ROADS
# ========================================

def draw_roads(draw, intersection_type: str):
    grass = "#79A96B"
    road = "#41464C"
    yellow = "#F4E6A0"
    white = "#FFFFFF"

    if intersection_type == "Four-Way":
        draw.rectangle((255, 0, 345, 500), fill=road)
        draw.rectangle((0, 205, 600, 295), fill=road)
```

```
        draw.line((300, 0, 300, 205),
                  fill=yellow, width=3)
        draw.line((300, 295, 300, 500),
                  fill=yellow, width=3)
        draw.line((0, 250, 255, 250),
                  fill=yellow, width=3)
        draw.line((345, 250, 600, 250),
                  fill=yellow, width=3)

        draw.line((255, 190, 300, 190),
                  fill=white, width=4)
        draw.line((300, 310, 345, 310),
                  fill=white, width=4)
        draw.line((240, 250, 240, 295),
                  fill=white, width=4)
        draw.line((360, 205, 360, 250),
                  fill=white, width=4)

    elif intersection_type == "T-Intersection":
        draw.rectangle((0, 205, 600, 295), fill=road)
        draw.rectangle((255, 250, 345, 500), fill=road)

        draw.line((0, 250, 255, 250),
                  fill=yellow, width=3)
        draw.line((345, 250, 600, 250),
                  fill=yellow, width=3)
        draw.line((300, 295, 300, 500),
                  fill=yellow, width=3)

        draw.line((300, 310, 345, 310),
                  fill=white, width=4)

    elif intersection_type == "Roundabout":
        draw.rectangle((255, 0, 345, 500), fill=road)
        draw.rectangle((0, 205, 600, 295), fill=road)

        draw.ellipse((185, 135, 415, 365),
                     fill=road)

        draw.ellipse((245, 195, 355, 305),
                     fill=grass)
```

```
        draw.ellipse((245, 195, 355, 305),
                     outline=white, width=3)

        draw.arc((215, 165, 385, 335),
                 0, 359, fill=yellow, width=2)

    if intersection_type != "T-Intersection":
        draw.text((355, 20), "NORTH", fill="black")

    draw.text((15, 175), "WEST", fill="black")
    draw.text((520, 175), "EAST", fill="black")
    draw.text((355, 465), "SOUTH", fill="black")



# ========================================
# VEHICLE COORDINATES
# ========================================

def vehicle_coordinates(vehicle: Vehicle,
                        intersection_type: str):

    position = vehicle.position

    if intersection_type == "Roundabout":
        # Approach the circular roadway.
        if position < 6:
            distance = (position + 3) * 17

            if vehicle.direction == "North":
                return (322, 20 + distance, "vertical")
            elif vehicle.direction == "South":
                return (278, 480 - distance, "vertical")
            elif vehicle.direction == "West":
                return (20 + distance, 272, "horizontal")
            else:
                return (580 - distance, 228, "horizontal")

        # Move around the circle.
        if vehicle.direction == "North":
            start_angle = -math.pi / 2
        elif vehicle.direction == "South":
```

```
            start_angle = math.pi / 2
        elif vehicle.direction == "West":
            start_angle = math.pi
        else:
            start_angle = 0

        angle = start_angle + (position - 6) * 0.35

        x = 300 + 85 * math.cos(angle)
        y = 250 + 85 * math.sin(angle)

        return (x, y, "horizontal")

    # Four-way and T-intersection.
    distance = (position + 3) * 17

    if vehicle.direction == "North":
        return (322, 20 + distance, "vertical")
    elif vehicle.direction == "South":
        return (278, 480 - distance, "vertical")
    elif vehicle.direction == "West":
        return (20 + distance, 272, "horizontal")
    else:
        return (580 - distance, 228, "horizontal")


# ========================================
# DRAW VEHICLES
# ========================================

def draw_vehicle(draw, vehicle: Vehicle,
                 intersection_type: str):

    if vehicle.position >= 12:
        return

    x, y, orientation = vehicle_coordinates(
        vehicle, intersection_type
    )

    if vehicle.waiting:
        color = "#E74C3C"
```

```
        else:
            color = "#3498DB"

        if orientation == "vertical":
            draw.rounded_rectangle(
                (x - 10, y - 17, x + 10, y + 17),
                radius=4,
                fill=color,
                outline="black",
                width=2
            )
            draw.rectangle(
                (x - 7, y - 8, x + 7, y - 2),
                fill="#BDE6F4"
            )
        else:
            draw.rounded_rectangle(
                (x - 17, y - 10, x + 17, y + 10),
                radius=4,
                fill=color,
                outline="black",
                width=2
            )
            draw.rectangle(
                (x - 6, y - 7, x, y + 7),
                fill="#BDE6F4"
            )

        draw.text(
            (x - 4, y - 5),
            str(vehicle.vehicle_id),
            fill="white"
        )


    def draw_intersection(state: State):
        picture = PILImage.new(
            "RGB", (600, 500), "#79A96B"
        )

        draw = ImageDraw.Draw(picture)
```

```
        draw_roads(draw, state.intersection_type)

        for vehicle in state.vehicles:
            draw_vehicle(
                draw,
                vehicle,
                state.intersection_type
            )

        return picture


    # ========================================
    # TRAFFIC MOVEMENT
    # ========================================

    def move_vehicles(state: State):
        # Vehicles move toward position 12.
        # Positions 6-8 represent the intersection.
        # One new vehicle may enter the center
        # during each step.

        center_occupied = False

        for vehicle in state.vehicles:
            if 6 <= vehicle.position < 9:
                center_occupied = True

        center_reserved = center_occupied

        # Vehicles nearest the exit move first.
        ordered = sorted(
            state.vehicles,
            key=lambda car: car.position,
            reverse=True
        )

        waiting_this_step = 0

        for vehicle in ordered:
            if vehicle.position >= 12:
                vehicle.waiting = False
```

```
            continue

        old_position = vehicle.position
        new_position = old_position + vehicle.speed

        if new_position > 12:
            new_position = 12

        # Prevent cars from passing others
        # traveling in the same direction.
        for other in state.vehicles:
            if other.vehicle_id != vehicle.vehicle_id:
                if other.direction == vehicle.direction:
                    if old_position < other.position < 12:
                        safe_position = other.position - 2

                        if new_position > safe_position:
                            new_position = safe_position

        # Stop at position 5 if the center
        # is occupied or already reserved.
        if old_position < 6 and new_position >= 6:
            if center_reserved:
                new_position = 5
            else:
                center_reserved = True

        # A vehicle must not move backward.
        if new_position < old_position:
            new_position = old_position

        vehicle.position = new_position

        if new_position == old_position:
            vehicle.waiting = True
            waiting_this_step += 1
        else:
            vehicle.waiting = False

    # Record waiting statistics.
    state.total_waiting_time += waiting_this_step
```

```
        if waiting_this_step > state.maximum_waiting:
            state.maximum_waiting = waiting_this_step


    # ========================================
    # STATISTICS
    # ========================================

    def calculate_results(state: State):
        moving = 0
        waiting = 0
        completed = 0

        for vehicle in state.vehicles:
            if vehicle.position >= 12:
                completed += 1
            elif vehicle.waiting:
                waiting += 1
            else:
                moving += 1

        total = len(state.vehicles)

        if total > 0:
            completion_rate = round(
                completed / total * 100, 1
            )

            average_delay = round(
                state.total_waiting_time / total, 2
            )
        else:
            completion_rate = 0
            average_delay = 0

        if state.simulation_step > 0:
            throughput = round(
                completed / state.simulation_step, 2
            )
        else:
            throughput = 0
```

```
        if average_delay < 2:
            congestion_level = "Low"
        elif average_delay < 5:
            congestion_level = "Moderate"
        else:
            congestion_level = "High"

        return [
            "Total Vehicles: " + str(total),
            "Completed Vehicles: " + str(completed),
            "Moving Vehicles: " + str(moving),
            "Waiting Vehicles: " + str(waiting),
            "Completion Rate: " + str(completion_rate) + "%",
            "Total Simulation Steps: " + str(state.simulation_step),
            "Total Waiting Time: " +
            str(state.total_waiting_time) + " vehicle-steps",
            "Average Delay Per Vehicle: " +
            str(average_delay) + " steps",
            "Maximum Vehicles Waiting at Once: " +
            str(state.maximum_waiting),
            "Throughput: " +
            str(throughput) + " vehicles per step",
            "Congestion Level: " + congestion_level
        ]


    # ========================================
    # PAGE 1 - HOME
    # ========================================

    @route
    def index(state: State) -> Page:
        return Page(state, [
            Header("Traffic Flow Simulator"),

            "Welcome to the Traffic Flow Simulator!",

            "Explore how vehicles move through "
            "different types of intersections.",

            "Choose an intersection, adjust traffic "
            "density and speed, and observe congestion.",
```

```
        Header("Available Intersections"),

        "Four-Way Intersection",
        "T-Intersection",
        "Roundabout",

        Button("Begin Simulation", "choose_intersection")
    ])


# ========================================
# PAGE 2 - CHOOSE INTERSECTION
# ========================================

@route
def choose_intersection(state: State) -> Page:
    return Page(state, [
        Header("Choose an Intersection"),

        "Select the intersection you want to study.",

        SelectBox(
            "intersection_choice",
            ["Four-Way", "T-Intersection", "Roundabout"],
            state.intersection_type
        ),

        Button("Continue", "save_intersection"),
        Button("Home", "index")
    ])


@route
def save_intersection(
        state: State,
        intersection_choice: str) -> Page:

    state.intersection_type = intersection_choice
    reset_data(state)

    return set_traffic(state)
```

```
# ========================================
# PAGE 3 - TRAFFIC SETTINGS
# ========================================

@route
def set_traffic(state: State) -> Page:
    return Page(state, [
        Header("Traffic Settings"),

        "Intersection: " + state.intersection_type,

        "Traffic Density:",

        SelectBox(
            "density_choice",
            ["Low", "Medium", "High"],
            density_name(state.traffic_density)
        ),

        "Vehicle Speed:",

        SelectBox(
            "speed_choice",
            ["Slow", "Normal", "Fast"],
            speed_name(state.vehicle_speed)
        ),

        Button("Save Settings", "save_traffic"),
        Button("Back", "choose_intersection")
    ])


@route
def save_traffic(
        state: State,
        density_choice: str,
        speed_choice: str) -> Page:

    if density_choice == "Low":
        state.traffic_density = 1
```

```
        elif density_choice == "Medium":
            state.traffic_density = 2
        else:
            state.traffic_density = 3

        if speed_choice == "Slow":
            state.vehicle_speed = 1
        elif speed_choice == "Normal":
            state.vehicle_speed = 2
        else:
            state.vehicle_speed = 3

        reset_data(state)

        return simulation(state)


    # ========================================
    # PAGE 4 - SIMULATION
    # ========================================

    @route
    def simulation(state: State) -> Page:

        content = [
            Header("Traffic Simulation"),

            "Intersection: " + state.intersection_type,

            "Traffic Density: " +
            density_name(state.traffic_density),

            "Vehicle Speed: " +
            speed_name(state.vehicle_speed),

            "Simulation Step: " +
            str(state.simulation_step)
        ]

        # Keep the main button above the picture.
        if state.simulation_completed:
            content.append("SIMULATION COMPLETE!")
```

```
            content.append(
                Button("View Final Results", "results")
            )

        elif state.simulation_running:
            content.append("Status: RUNNING")

            content.append(
                Button("Advance Simulation", "advance_simulation")
            )

        else:
            content.append("Status: READY")

            content.append(
                Button("Start Simulation", "start_simulation")
            )

        content += [
            Button("Reset Simulation", "reset_simulation"),
            Button("View Results", "results"),
            Button("Change Settings", "set_traffic"),
            Button("Home", "index"),

            Header("Intersection Visualization"),

            Image(
                draw_intersection(state),
                600,
                500,
                alt="Overhead traffic intersection simulation"
            ),

            "Blue = Moving Vehicles",
            "Red = Waiting Vehicles"
        ]

        return Page(state, content)


    # ========================================
```

```
# START SIMULATION
# ========================================

@route
def start_simulation(state: State) -> Page:

    reset_data(state)

    create_vehicles(state)

    state.simulation_running = True

    return simulation(state)


# ========================================
# ADVANCE SIMULATION
# ========================================

@route
def advance_simulation(state: State) -> Page:

    if state.simulation_running:
        move_vehicles(state)

        state.simulation_step += 1

        all_finished = True

        for vehicle in state.vehicles:
            if vehicle.position < 12:
                all_finished = False

        if all_finished:
            state.simulation_running = False
            state.simulation_completed = True

    return simulation(state)


# ========================================
# RESET SIMULATION
```

```
# ========================================

@route
def reset_simulation(state: State) -> Page:

    reset_data(state)

    return simulation(state)


# ========================================
# PAGE 5 - RESULTS
# ========================================

@route
def results(state: State) -> Page:

    content = [
        Header("Traffic Simulation Results"),

        "Intersection: " + state.intersection_type,

        "Traffic Density: " +
        density_name(state.traffic_density),

        "Vehicle Speed: " +
        speed_name(state.vehicle_speed)
    ]

    if state.simulation_completed:
        content.append(
            Header("Simulation Successfully Completed!")
        )
        content.append(
            "All vehicles have exited the intersection."
        )

    elif state.simulation_running:
        content.append(
            "Simulation is still running. "
            "These are the current statistics."
        )
```

```
        else:
            content.append(
                "Start a simulation to generate results."
            )

        content.append(Header("Traffic Statistics"))

        statistics = calculate_results(state)

        for statistic in statistics:
            content.append(statistic)

        if state.simulation_completed:
            content.append(Header("Final Analysis"))

            average_delay = 0

            if len(state.vehicles) > 0:
                average_delay = (
                    state.total_waiting_time /
                    len(state.vehicles)
                )

            if average_delay < 2:
                content.append(
                    "Traffic flowed efficiently through "
                    "the intersection with minimal delays."
                )

            elif average_delay < 5:
                content.append(
                    "The intersection experienced moderate "
                    "congestion and some vehicle delays."
                )

            else:
                content.append(
                    "The intersection experienced high "
                    "congestion and significant delays."
                )
```

```
            content.append(
                "Try changing the intersection type, "
                "traffic density, or vehicle speed to "
                "compare the results."
            )

        content += [
            Button("Return to Simulation", "simulation"),
            Button("Reset Simulation", "reset_simulation"),
            Button("Change Intersection", "choose_intersection"),
            Button("Home", "index")
        ]

        return Page(state, content)


# ========================================
# START WEBSITE
# ========================================

initial_state = State(
    "Four-Way",
    1,
    [],
    False,
    0,
    2,
    False,
    0,
    0
)

start_server(
    initial_state,
    site_title="Traffic Flow Simulator"
)
```

What your final results page will look like

For example, if you run a simulation with 8 vehicles, you might see results like these. These are illustrative numbers, not actual output from a tested run.

Simulation Complete!

Four-Way Intersection · Medium Traffic · Normal Speed

Vehicles completed	Completion rate
8 / 8	100%
Simulation steps	**Total waiting time**
18	12

Average delay per vehicle 1.5 steps

Throughput 0.44 vehicles/step

Final analysis: Traffic flowed efficiently through the intersection with minimal delays.

How the new statistics work

Statistic	What it measures
Total waiting time	Combined number of steps vehicles spent stopped
Average delay	Total waiting time divided by the number of vehicles
Maximum waiting	Largest number of stopped vehicles during a single step
Throughput	Completed vehicles divided by simulation steps
Completion rate	Percentage of vehicles that finished

These are simplified simulation measurements, not real-world traffic engineering estimates. The current roundabout also uses the same basic intersection-conflict rules as the other layouts, so the results aren't yet a realistic comparison of intersection designs.

One important thing: I haven't been able to execute this in a live Drafter installation, so test it in Thonny. The next improvement I'd recommend is giving the three intersection types different traffic movement rules. That would make your results more meaningful, because you'd be able to compare how each design handles the same amount of traffic rather than mainly comparing density and speed.

You said:

now make docstrings to tell how things work

ChatGPT said:

Here are **docstrings for every function in your Traffic Flow Simulator**, written in a straightforward style appropriate for your Python course.

You can copy each docstring and place it **immediately below the corresponding `def` line**, before any code in the function.

I'll use triple quotation marks (`"""`) and explain what each function does, what information it uses, and what it returns.

1. Dataclasses

```
@dataclass
class Vehicle:
    """Represents one vehicle in the traffic simulation.

    Stores the vehicle's ID, position, speed, direction,
    and whether it is waiting in traffic.
    """
```

```
@dataclass
class State:
   """Converts the traffic density number into its name.

1 represents Low, 2 represents Medium, and 3 represents High.
Returns the density as a string.
"""
```

2. Helper functions

`density_name`

```
"""Converts the traffic density number into its name.
```

```
1 represents Low, 2 represents Medium, and 3 represents High.
Returns the density as a string.
"""


 speed_name


"""Converts the vehicle speed number into its name.

1 represents Slow, 2 represents Normal, and 3 represents Fast.
Returns the speed as a string.
"""


 reset_data


"""Resets the simulation without changing the user's settings.

Removes all vehicles and resets the simulation status,
step counter, and waiting statistics.
"""


 create_vehicles


"""Creates vehicles based on the selected traffic density.

Generates 4, 8, or 12 vehicles depending on the density.
Assigns each vehicle an ID, direction, starting position,
and speed, then adds it to the State's vehicles list.
"""
```

3. Drawing functions

```
 draw_roads


"""Draws the selected intersection layout.

Uses Pillow to draw the roads, lane markings, and
direction labels for a four-way intersection,
```

```
T-intersection, or roundabout.
"""
```

```
vehicle_coordinates
```

```
"""Calculates where a vehicle should appear in the image.

Uses the vehicle's position, direction, and intersection
type to calculate its x and y coordinates.
Returns the coordinates and vehicle orientation.
"""
```

```
draw_vehicle
```

```
"""Draws an individual vehicle on the intersection image.

Uses the vehicle's coordinates to draw a numbered car.
Moving vehicles are blue and waiting vehicles are red.
Vehicles that have completed their route are not drawn.
"""
```

```
draw_intersection
```

```
"""Creates the complete intersection visualization.

Creates a Pillow image, draws the selected road layout,
and loops through the vehicles to draw each car.
Returns the completed image for display on the website.
"""
```

4. Traffic simulation

```
move_vehicles
```

```
"""Advances every vehicle by one simulation step.

Loops through vehicles and updates their positions based
on their speed. Checks for vehicles ahead and whether
```

```
the intersection is occupied to prevent some conflicts.
Records the total and maximum number of waiting vehicles.
"""
```

5. Statistics

```
calculate_results
```

```
"""Calculates statistics from the current simulation.

Counts moving, waiting, and completed vehicles.
Calculates the completion rate, average delay,
throughput, and congestion level.
Returns a list of strings containing the statistics.
"""
```

6. Website pages and routes

```
index
```

```
"""Displays the homepage of the Traffic Flow Simulator.

Explains the purpose of the website, lists the available
intersections, and provides a button to begin.
Returns the homepage as a Drafter Page.
"""
```

```
choose_intersection
```

```
"""Displays the intersection selection page.

Allows the user to choose a four-way intersection,
T-intersection, or roundabout using a dropdown menu.
Returns the selection page.
"""
```

```
save_intersection
```

```
    """Saves the user's selected intersection type.

    Updates the State with the chosen intersection,
    resets the previous simulation, and opens
    the traffic settings page.
    """


    set_traffic


    """Displays the traffic settings page.

    Allows the user to choose traffic density and
    vehicle speed using dropdown menus.
    Returns the traffic settings page.
    """


    save_traffic


    """Saves the user's traffic density and speed settings.

    Converts the selected names into integer values,
    updates the State, resets the simulation, and
    opens the simulation page.
    """


    simulation


    """Displays the traffic simulation page.

    Shows the selected settings, simulation status,
    current step, and overhead intersection image.
    Provides buttons to start, advance, reset,
    and navigate to other pages.
    Returns the simulation page.
    """


    start_simulation
```

```
    """Starts a new traffic simulation.

    Resets previous simulation data, creates vehicles
    based on the selected settings, and changes the
    simulation status to running.
    Returns the updated simulation page.
    """


    advance_simulation


    """Advances the traffic simulation by one step.

    Moves vehicles and increases the step counter
    when the simulation is running. Checks whether
    all vehicles have finished and marks the
    simulation complete when they have.
    Returns the updated simulation page.
    """


    reset_simulation


    """Resets the current traffic simulation.

    Clears the vehicles, step counter, completion
    status, and traffic statistics while keeping
    the user's intersection and traffic settings.
    Returns the reset simulation page.
    """


    results


    """Displays the simulation results page.

    Shows the selected settings and calculated traffic
    statistics. If the simulation is complete, it also
    provides an analysis of the congestion level.
    Returns the results page.
    """
```

Example of correct placement

Here's how your `create_vehicles` function should look with its docstring:

```
def create_vehicles(state: State):
    """Creates vehicles based on the selected traffic density.

    Generates 4, 8, or 12 vehicles depending on the density.
    Assigns each vehicle an ID, direction, starting position,
    and speed, then adds it to the State's vehicles list.
    """
    state.vehicles = []

    if state.traffic_density == 1:
        amount = 4
    elif state.traffic_density == 2:
        amount = 8
    else:
        amount = 12

    # Rest of your existing function...
```

These docstrings cover **all 19 functions** in your current program, plus both dataclasses. They explain the purpose and behavior of each function without being unnecessarily technical.

You can paste them into your existing code without changing how the program runs.